Lecture 26.05: Linear Regression GD (Step 3 — Gradients Calculation)

📌 Topic Overview

In this lecture, Shraddha ma'am covers **Step 3 of Gradient Descent**: deriving the partial derivatives of the MSE loss function with respect to weights w and bias b using multivariable calculus chain rule.

📐 Mathematical Derivation

Starting with the MSE cost function:

$$J(w,b) = \frac{1}{2m}\sum_{i=1}^{m}\left((X^{(i)}w + b) - y^{(i)}\right)^2$$

1. Partial Derivative with respect to Weights w_j:

By the chain rule:

$$\frac{\partial J}{\partial w} = \frac{1}{m}X^T(\hat{y} - y)$$

2. Partial Derivative with respect to Bias b:

$$\frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}^{(i)} - y^{(i)})$$

These gradients point in the direction of steepest ascent; moving in the **opposite direction** ($-\nabla J$) guarantees reduction in error.

💻 Python Implementation

```
import numpy as np

def compute_linear_regression_gradients(X, y_true, y_pred):
    """
    Step 3 of Gradient Descent: Compute partial derivatives of MSE cost.
    Chain Rule Derivatives:
    dw = dJ/dw = (1 / m) * X^T * (y_pred - y_true)
    db = dJ/db = (1 / m) * sum(y_pred - y_true)
    """
    m = len(y_true)
    error = y_pred - y_true

    # Vectorized gradient computation
    dw = (1.0 / m) * np.dot(X.T, error)
    db = (1.0 / m) * np.sum(error)

    return dw, db

# --- Function Call & Testing ---
X_test = np.array([[1.0, 2.0], [2.0, 1.0], [3.0, 4.0]])
y_actual = np.array([5.0, 4.0, 11.0])
y_predicted = np.array([4.5, 3.8, 10.2])

dw_grad, db_grad = compute_linear_regression_gradients(X_test, y_actual, y_predict
print("Weight Gradients (dw):", dw_grad.round(4))
print("Bias Gradient (db):   ", round(db_grad, 4))
```

🔑 Key Takeaways

1. Gradients indicate the magnitude and direction of the steepest error slope.
2. The matrix multiplication $X^T(\hat{y} - y)$ simultaneously computes the gradients for all d features in a single vectorized operation.